

CAPL

一、CAPL语言的概述和基础

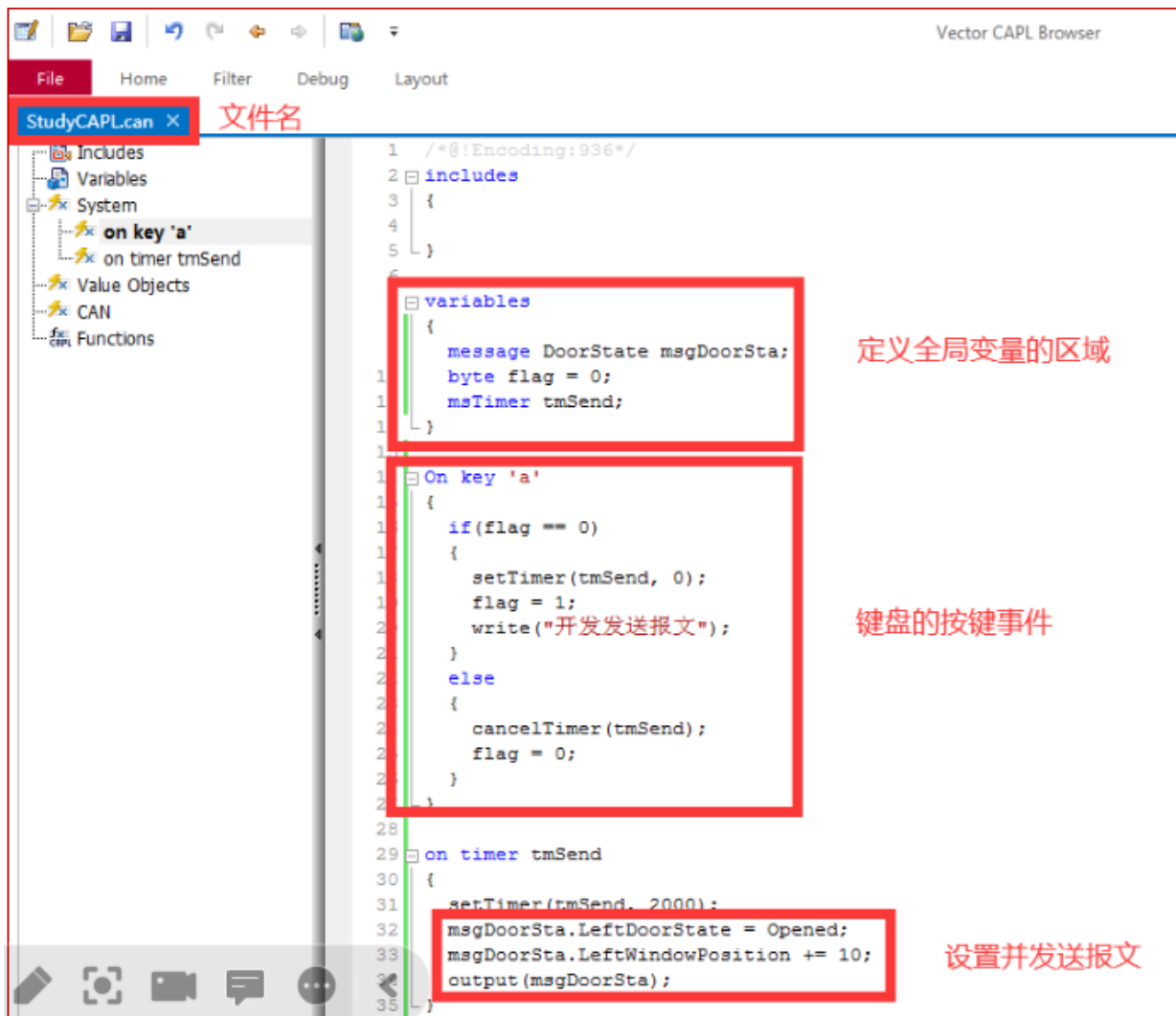
1. CAPL的概述

- CAPL (Communication Access Programming Language) —— 通信访问编程语言
- 专为CANoe工具链定制，是一种类C语法的语言，是一种事件驱动的语言

2. CAPL的作用

- ECU的仿真实现（报文的收发、数据处理）
- 实现总线日志记录的控制
- 实现总线数据的自动化分析
- 实现ECU功能的自动化测试
- 实现ECU报文发送周期的自动化测试
- 实现诊断测试的自动化

3. 代码的框架结构



4. 简单的数据类型

- 数值类型：整型、浮点型
 - 整型
 - 无符号：byte 、 word 、 dword 、 gword
 - 有符号：int 、 long 、 int64
 - 浮点型
 - float
 - double

整型	有/无符号	长度	范 围	备 注
byte	unsigned	1B	0~255	字节，8 位，无符号
word	unsigned	2B	0~65 535	字，16 位，无符号
dword	unsigned	4B	0~4 294 967 295	双字，32 位，无符号
gword	unsigned	8B	0~18 446 744 073 709 551 615	整型，64 位，无符号
int	signed	2B	-32 768~32 767	短整型，16 位
long	signed	4B	-2 147 483 648~2 147 483 647	长整型，32 位
int64	signed	8B	-9 223 372 036 854 775 808 ~9 223 372 036 854 775 807	整型，64 位

浮 点 型	长 度	范 围	描 述
float	4B	$1.7 \times 10^{-308} \sim 1.7 \times 10^{+308}$	单精度
double	8B	$1.7 \times 10^{-308} \sim 1.7 \times 10^{+308}$	双精度

- 字符类型
 - char：使用单引号括起来的单个字符，可以使用转义字符
 - 字符类型可以跟整型直接相互做算术运算
 - 每一个字符可以看成其对应码值的整数

ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符	ASCII值	控制字符
0	NUL	32	(space)	64	@	96	,
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	,	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

5. 变量

- 先声明变量，才能使用
- 在CAPL中，有全局变量和局部变量之分
 - 定义在variables的代码块中的变量称为全局变量，可以被这个文件中的所有其它代码块中的语句使用。
 - variables代码块中只能用来定义全局变量，不能写其它语句
 - 定义在其它代码块中的变量是局部变量，局部变量必须声明在代码块的最开始。
- CAPL中的变量是强类型的，定义变量时必须指定该变量的数据类型，而且无法更改

```
// 数据类型 变量名;
int age;
// 数据类型 变量名 = 值;
float tall = 1.81;
```

- 在声明变量的时候，变量前使用const关键字，该变量就变成了常量，一旦赋值，就无法改变。
- 一般声明常量的时候，习惯全大写

```
const float YZL = 3.1415926;
write("圆周率为: %lf", YZL);
YZL = 3.14 // 会报错
```

6. 输出函数write和不同类型的数据的占位符

`write("我叫%s，今年%5d岁，我身高%f。", "张无忌", 23, 1.836);`

控制符	说明
%d 整数	按十进制整型数据的实际长度输出。
%ld	输出长整型数据。
%md	m 为指定的输出字段的宽度。如果数据的位数小于 m，则左端补以空格，若大于 m，则按实际位数输出。
%u	输出无符号整型（unsigned）。输出无符号整型时也可以用 %d，这时是将无符号转换成有符号数，然后输出。但编程的时候最好不要这么写，因为这样要进行一次转换，使 CPU 多做一次无用功。
%c 字符	用来输出一个字符。
%f 小数	用来输出实数，包括单精度和双精度，以小数形式输出。不指定字段宽度，由系统自动指定，整数部分全部输出，小数部分输出 6 位，超过 6 位的四舍五入。
%mf	输出实数时小数点后保留 m 位，注意 m 前面有个点。
%o	以八进制整数形式输出，这个就用得很少了，了解一下就行了。
%s 字符串	用来输出字符串。用 %s 输出字符串同前面直接输出字符串是一样的。但是此时要先定义字符数组或字符指针存储或指向字符串，这个稍后再讲。
%x (或 %X 或 %#x 或 %#X)	以十六进制形式输出整数，这个很重要。

7. 复合数据类型 —— 数组

- 由一组简单数据类型组成的数据
 - `int arr[3] = {1, 2, 3};` —— 声明、开辟空间、初始化
 - `int arr2[5];` —— 声明了int类型的数组arr2、开辟了5个位置的空间
 - `char arr3[10];` —— 声明了char类型的数组arr3、开辟了10个位置的空间
- 数组有一维的，也有二维的
 - `int scores[3][5];`
- 可以使用字符串常量 “任意文本” 给字符数组变量赋值，但注意此时的字符数组的长度应该大于或等于字符串常量中的字符个数（如果是中文，个数要乘以2） + 1
 - `char name[5] = "Jack";`
 - `char name2[5] = {'J', 'a', 'c', 'k', '\0'};`
 - `char name3[7] = "张无忌";`
- 使用 `elCount` 内置函数，来获取数组的长度
 - `write("参加考试的小组数量: %d", elCount(scores));`

8. 复合数据类型 —— 枚举

枚举示例:

- ```
enum Sex { Male, Female};
enum Direction { Up, Down, Left, Right} my;
enum Direction your;
enum Direction her = Left;
enum Direction his = right;
```

## 9. 复合数据类型 —— 结构

### 结构示例:

- ```
struct Distance {  
    float lng;  
    float lat;  
};  
struct Distance veh = {lng=27.35, lat=0.2};  
struct Point {int x; int y;} p1 = {x=83, y=92};  
struct Point p2 = {x=77, y=93};  
struct Point p3 = {88, 63};  
struct {int x; int y; int z; } p4 = {3, 2, 1};
```

10. 特殊数据类型 —— 报文

使用关键字**message**来声明一个报文变量，当使用**message**声明报文变量时，默认变量为**CAN**报文变量。当有数据库支撑的时候，一个完整的声明应该包括 **message ID** 或者 **message name**。

创建一个报文类型的变量示例如下:

```
message 0x666 msg1;  
message DoorState msg2;  
message 0x30F msg3;  
message 200 msg4;  
message * msg5;
```

报文变量的属性:

```
msg1.id、msg1.name、msg1.dir、msg1.dlc、msg1.LeftDoorState  
msg1.byte(N)
```

输出报文到**CAN**总线:

- ```
output(msg1);
```

## 11. 特殊数据类型 —— 定时器

定时器变量会绑定到该定时器的事件处理函数，当开启该定时器时，会在经过指定的时长后，执行处理函数中的代码。

CAPL 提供了两种定时器变量。

- 以秒为单位的定时器类型：*Timer*
- 以毫秒为单位的定时器类型：*msTimer*

以毫秒定时器为例：

```
variables
{
 msTimer tmr1; // 声明一个毫秒定时器tmr1
}

on timer tmr1 // 为tmr1定时器绑定一个处理函数
{
 write("迟到的爱");
}

on key 'a'
{
 setTimer(tmr1, 3000); // 开始定时器，3秒后执行处理函数中的代码
}
```

12. 运算符

| 数学/关系运算符           |                               |       |
|--------------------|-------------------------------|-------|
| +, -, *, /, %      | 加, 减, 乘, 除, 余数 (取模)           | 双目运算符 |
| ==, !=             | 等于, 不等于                       | 双目运算符 |
| >, >=, <, <=       | 大于, 大于等于, 小于, 小于等于            | 双目运算符 |
| ++, --             | 自增运算符, 自减运算符                  | 单目运算符 |
| =                  | 赋值运算符                         |       |
| +=, -=, *=, /=, %= | 加后赋值, 减后赋值, 乘后赋值, 除后赋值, 取模后赋值 |       |
| 逻辑运算符              |                               |       |
| &&,                | 逻辑与, 逻辑或                      | 双目运算符 |
| ?:                 | 条件运算符                         | 三目运算符 |
| !                  | 逻辑非运算符                        | 单目运算符 |
| 位运算                |                               |       |
| ~                  | 按位取反运算符                       | 单目运算符 |
| &, ^,              | 按位与, 按位异或, 按位或                |       |
| <<, >>             | 左移, 右移                        |       |
| <<=, >>=           | 左移后赋值, 右移后赋值                  |       |
| &=, ^=,  =         | 按位与后赋值, 按位异或后赋值, 按位或后赋值       |       |

13. 流程控制结构

- 分支结构

单分支if结构:

```
if(条件){
 主体
}
```

双分支if-else结构:

```
if(条件){
 主体
} else {
 主体
}
```

多分支if-else if-else结构:

```
if(条件){
 主体
} else if(条件){
 主体
}
else {
 主体
}
```



- 循环结构

while循环结构:

```
while(条件){
 主体
}
```

do-while循环结构:

```
do {
 主体
} while(条件);
```

for循环结构:

```
for(初始化; 条件; 增量){
 主体
}
```

•

switch开关结构:

```
switch(表达式){

 case 常量1:
 主体
 break;

 case 常量2:
 主体
 break;

 case 常量N:
 主体
 break;

 default:
 主体
 break;
}
```

循环流程控制关键字:

```
break
continue
```

## 14. 自定义函数

# 语法示例

*返回值类型 函数名(形参列表)*

*{*

*函数主体语句*

*使用return关键返回数据*

*如果返回值类型为void，可以省略return*

*}*

•

## 二、CAPL中常用的内置函数

### 1. 输出到Write窗口的相关函数

- `write` —— 输出任意文本信息到Write窗口的CAPL标签页上
- `writeLineEx` —— 输出任意文本信息到Write窗口中的指定标签页上，并指定消息状态
  - `writeLineEx(4, 1, "Hello")` —— 输出到Test标签页（4）上，消息状态为：普通（1）
  - `writeLineEx(4, 2, "Hello")` —— 输出到Test标签页（4）上，消息状态为：警告（2）
- `setWriteDbgLevel` —— 设置调试信息（输出到System标签页上的）的输出优先级
  - `setWriteDbgLevel(3);` 将调试信息输出的优先级设置为 3
- `writeDbgLevel` —— 输出调试信息到Write窗口的System标签也上，只输出满足优先级的
  - `writeDbgLevel(1, "Hello");` 以1这个优先级输出调试信息 "Hello"

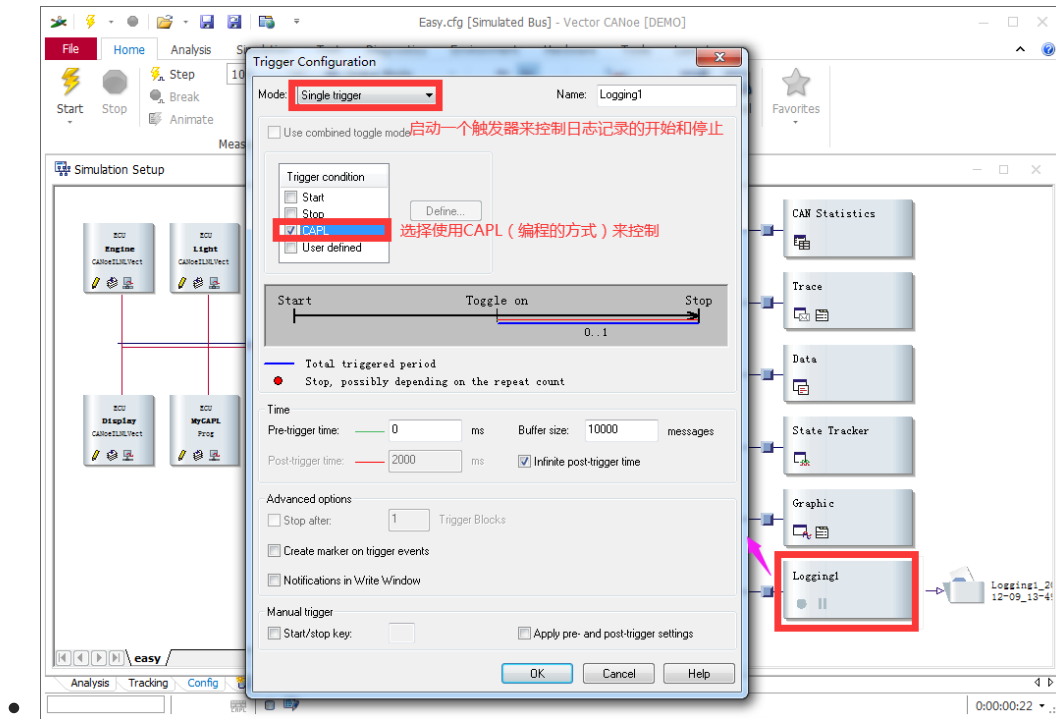
### 2. 总线控制的相关函数

- `stop` —— 停止程序（停止了总线上的测量）

### 3. 日志（Logging）控制的相关函数

- 先对日志模块进行配置（开始使用CAPL编程的方式控制日志的开启和停止）





- `setLogFileName` —— 设置日志模块的文件存放位置和名称（含有格式）
  - 可以指定具体为哪个日志模块设置
  - `setLogFileName("LoggingAEB", "Logging/{LoggingBlock}_{MeasurementStart}.asc");`
- `startLogging` —— 启动日志的记录
  - `startLogging()` —— 启动所有日志模块的记录
  - `startLogging("LoggingName")` —— 启动指定日志模块的记录
- `stopLogging` —— 停止日志的记录
  - `stopLogging()` —— 停止所有日志模块的记录
  - `stopLogging("LoggingName")` —— 停止指定日志模块的记录

#### 4. 定时器控制的相关函数

- `setTimer(定时器变量, 时长)` —— 启动一个定时器，在指定的“时长”后开始执行定时器的处理函数
- `cancelTimer(定时器变量)` —— 取消一个定时器

#### 5. 文件读写的相关函数

|                            |           |
|----------------------------|-----------|
| <code>openFileWrite</code> | 以写入模式打开文件 |
| <code>openFileRead</code>  | 以读取模式打开文件 |
| <code>filePutString</code> | 输出字符串到文件中 |
| <code>fileGetString</code> | 从文件中读取字符串 |
| <code>fileClose</code>     | 关闭文件      |

```

int fh;

fh = openFileWrite("文件的路径", 0); // 覆盖写入
if(fh) // 代表文件成功打开
{
 filePutString("贾老师真帅! \n", 100, fh); // 写入籽富含窠
 filePutString("真的真的帅! ", 100, fh); // 写入字符串

 fileClose(fh); // 关闭文件
}

int fileHandle;
char line[200];

// 步骤1: 打开文件 (以读取文本字符的模式)
fileHandle = openFileRead("文件路径", 0);
if(fileHandle)
{
 // 步骤2: 从文件中读取字符串
 while(fileGetString(line, elCount(line), fileHandle))
 {
 write("读取的一行内容: %s", line);
 }
 // 步骤3: 关闭文件
 fileClose(fileHandle);
}

```

## 6. 数学类的函数

- `_round` —— 四舍五入
- `_ceil` —— 向上取整
- `_floor` —— 向下取整
- `random` —— 产生指定范围的随机整数
- `sin` —— 求正弦值
- `cos` —— 求余弦值

## 7. 其它的常用函数

- `elCount` —— 计算数组、字符串的长度 (在计算字符串长度时要格外注意)
- `getLocalTimeString` —— 获取系统当前的时间字符串
- `snprintf` —— 将格式化的数据写入字符串

```

char buffer[100]; // 写入到buffer字符串中
int age = 23;
char name[10] = "Jack";
snprintf(buffer, elCount(buffer), "我叫%s, 今年%d岁。", name, age);

```

## 三、常用的事件

- CAPL是事件驱动的语言，常用的事件如下

| 事件     | 语法                          |
|--------|-----------------------------|
| 程序启动事件 | <b>on start</b>             |
| 测量结束事件 | <b>on stopMeasurement</b>   |
| 按键事件   | <b>on key 'k'</b>           |
| 定时器事件  | <b>on timer</b> 定时器变量名      |
| 报文事件   | <b>on message</b> 报文名或ID    |
| 信号事件   | <b>on signal</b> 信号名        |
| 信号更新事件 | <b>on signal_update</b> 信号名 |

- **在on key事件发生时**

- `this` 可用，代表当前按的是哪个键的字符数据

- **on message事件**

- 当在总线上发现一个指定的报文出现时触发该事件
- 每次总线上探测到这个报文，就会触发一次。
- 该事件发生时，`this` 关键字可用，代表的就是发现的那个报文数据，数据类型是 `message` 变量。
  - 可以 `this.id` 、 `this.name` 、 `this.dir` 、 `this.dlc` 、 `this.信号名`
- 经常会关注某个报文的变化，因此会在这类事件发生时，进行业务处理。

```
// 当总线上发现了ID为0x123的报文
on message 0x123 { }
```

```
// 当总线上发现了名为EngineState的报文（有数据库的支持）
on message EngineState { }
```

- **on signal事件**

- 当信号变化时触发
- 该事件发生时，`this` 关键字可用，代表这个信号的当前值，数据类型是 `float`

- **on signal\_update 事件**

- 当信号更新时（频率就是该信号所属报文发出的频率）触发
- 该事件发生时，`this` 关键字可用，代表这个信号的当前值，数据类型是 `float`

## 四、访问信号的值

- 在任意时刻访问当前总线上出现的任意信号的当前值

- 语法 —— `$信号名` 、 `$报文名::信号名`
- 用法
  - 获取信号的物理量（值）
    - `$信号名`

- \$信号名.phys
- 获取信号的原始值
- \$信号名.raw

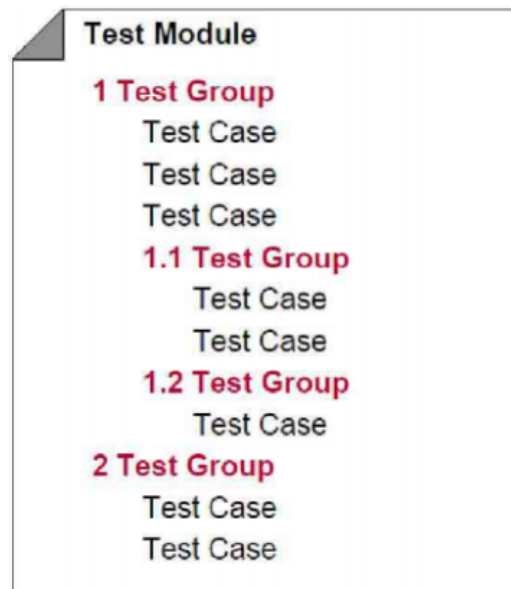
```
On signal HeadLight
{
write("信号HeadLight发生了变化, 信号的值为: %d", (int)this);
// 使用【$信号名】来获取信号的物理量
write("此时FlashLight信号的值为: %d", (int)$FlashLight);
write("此时引擎的OnOff信号的值为: %d", (int)$OnOff);
write("此时引擎的EngineSpeed信号的值为: %d", (int)$EngineSpeed);
// 使用【$信号名.phys】来获取信号的物理量
write("此时引擎的EngineSpeed信号的物理量为: %d", (int)$EngineSpeed.phys);
// 使用【$信号名.raw】来获取信号的原始值
write("此时引擎的EngineSpeed信号的原始值为: %d", (int)$EngineSpeed.raw);
}
```

## 五、测试模块 Test Module

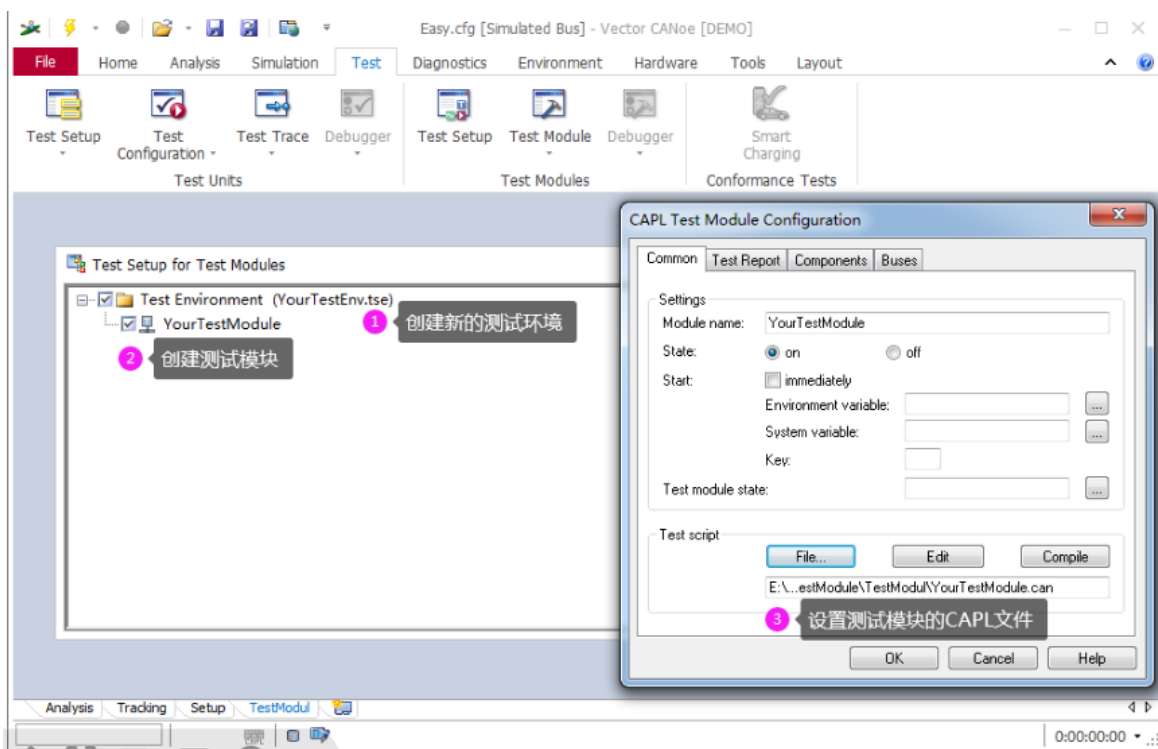
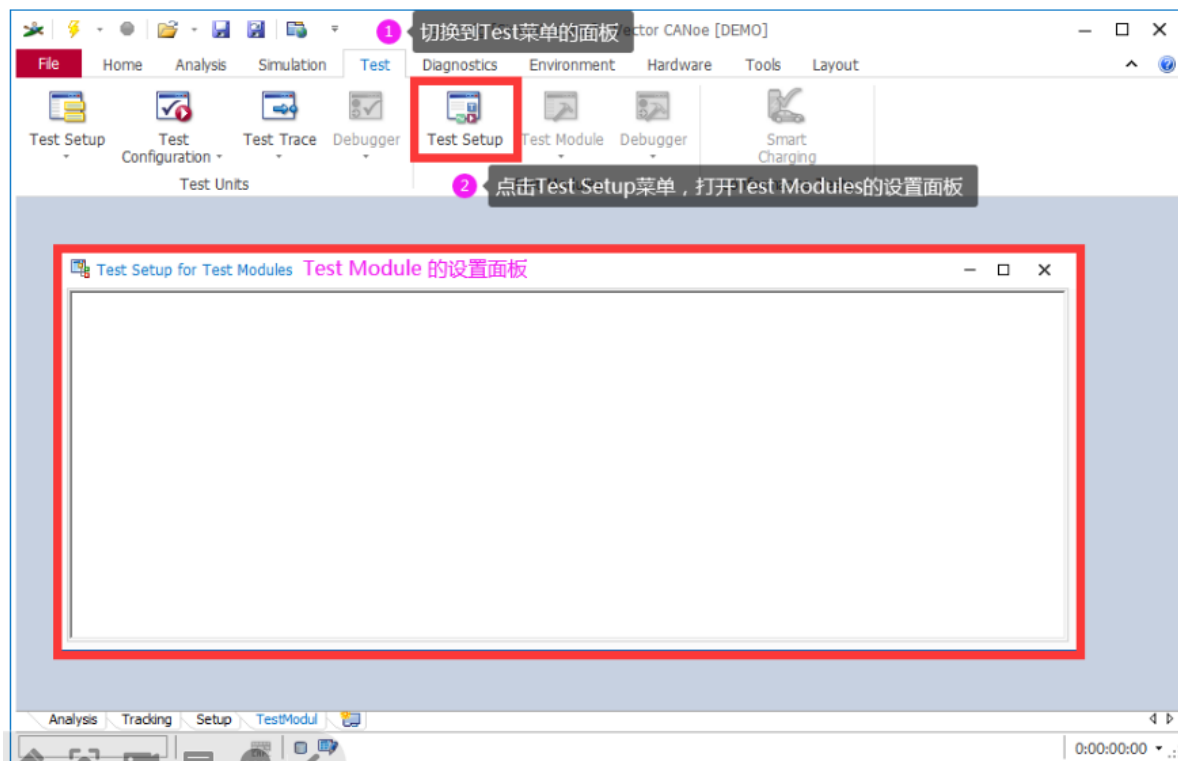
### 1. Test Module的结构

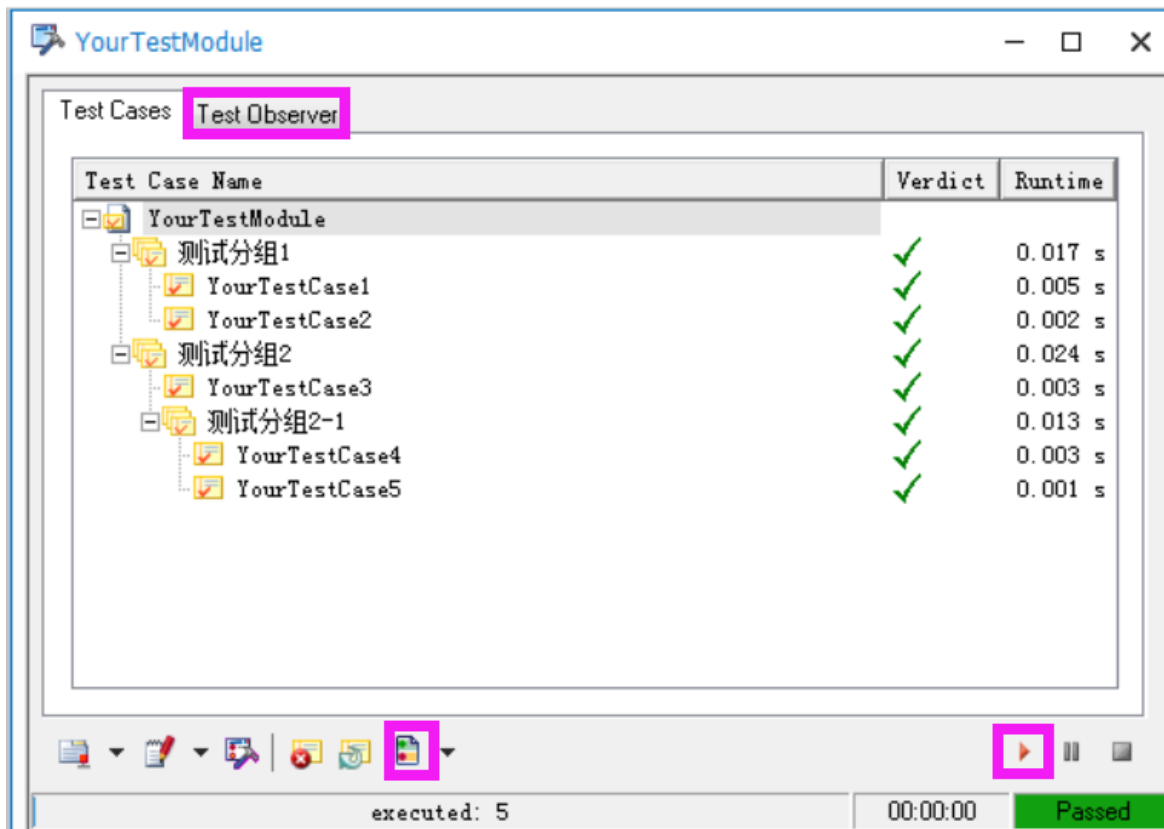
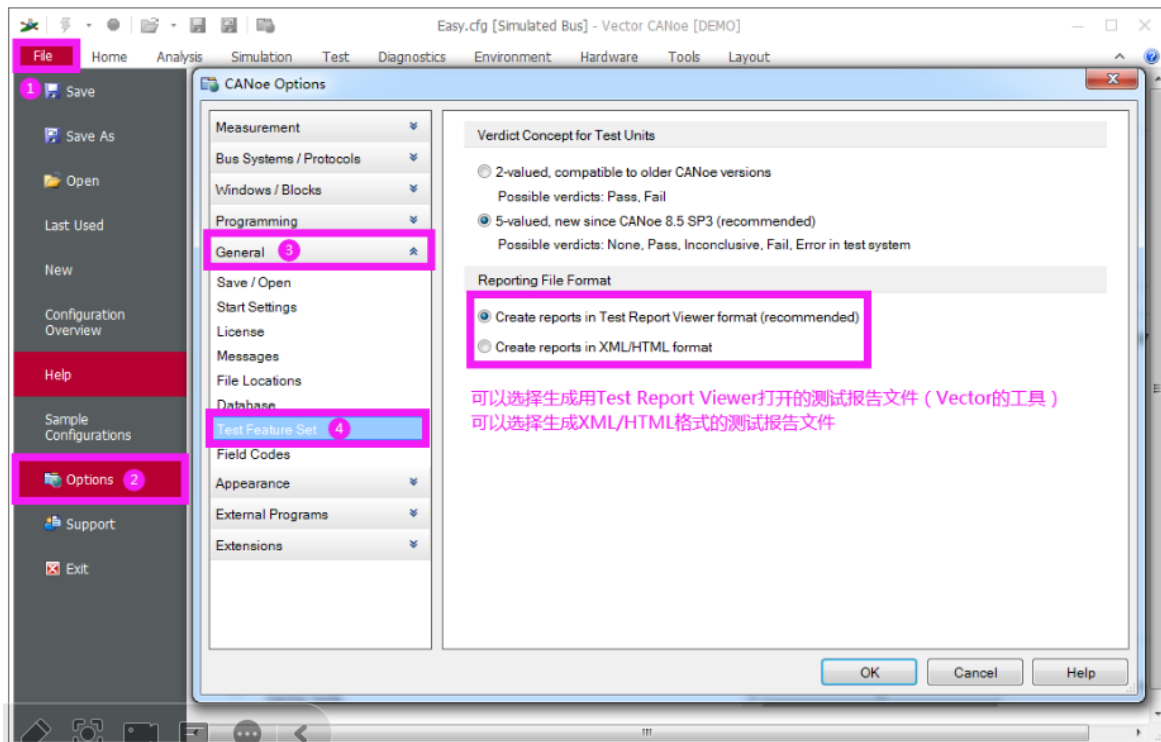
#### Test Module的架构

- 一个测试模块中可以包含多个测试用例
- 一个测试用例中可以包含多个测试步骤
- 可以使用测试组对测试用例进行分类
- 测试组可以嵌套



### 2. 在CANoe中创建并配置Test Module组件





### 3. Test Module的CAPL脚本代码的基本结构

- 要创建.can文件跟TestModule进行关联
- 脚本文件中必须要有 `MainTest` 函数，用于描述整个测试模块中的执行流程
- 要编写用例，必须定义测试用例函数，使用 `testcase` 关键字定义

```

MainTest()
{
 MyTestCase1();
 MyTestCase2();
}

testcase MyTestCase1()
{
}

testcase MyTestCase2()
{
}

```

#### 4. 常用函数 —— “测试架构”相关的函数

| 函数名                   | 作用                      |
|-----------------------|-------------------------|
| TestModuleTitle       | 设定测试模块的标题               |
| TestModuleDescription | 设定测试模块的描述信息             |
| TestGroupBegin        | 设定测试分组的开始，并指定分组的标题和描述信息 |
| TestGroupEnd          | 设定测试分组的结束               |
| TestCaseTitle         | 设定测试用例的标题               |
| TestCaseDescription   | 设定测试用例的描述信息             |
| TestStep              | 设定测试步骤的信息，不会对测试结果产生影响   |
| TestStepPass          | 设定测试步骤的结果通过，符合预期        |
| TestStepFail          | 设定测试步骤的结果失败             |

#### 5. 常用函数 —— “等待”相关的函数

| 函数名                           | 作用           |
|-------------------------------|--------------|
| TestWaitForTimeout            | 等待直到超时       |
| TestWaitForSignalMatch        | 等待信号出现并等于预期值 |
| testWaitForTesterConfirmation | 等待用户确认弹出的对话框 |

#### 6. 常用函数 —— “信号”相关的函数

| 函数名                            | 作用                        |
|--------------------------------|---------------------------|
| CheckSignalInRange             | 检查信号是否在指定范围               |
| TestValidateSignalInRange      | 检查信号是否在指定范围，并将判定结果写入测试报告  |
| TestValidateSignalOutsideRange | 检查信号是否超出指定范围，并将判定结果写入测试报告 |
| CheckSignalMatch               | 检查信号是否等于预期值               |
| TestValidateSignalMatch        | 检查信号是否等于预期值，并将判定结果写入测试报告  |

#### 7. 常用函数 —— “测试报告”相关的函数

| 函数名                       | 作用              |
|---------------------------|-----------------|
| TestReportAddEngineerInfo | 向测试报告中添加测试员的信息  |
| TestReportAddSUTInfo      | 向测试报告中添加被测系统的信息 |
| testReportAddSetupInfo    | 向测试报告中添加测试设置的信息 |

## 8. 常用函数 —— “探测检查”相关的函数

- 为了完成一段时间内的持续探测检查，并能够统计这段时间内的各项数据，并且获取检查的结果
  - 先：调用一个具体规则的“开始检查函数”，返回一个检查的标识ID
  - 再：等待一段指定的时长（这个过程中），系统会持续探测检查
  - 最后：判定检查的探测结果（违反规则的事件的发生次数），是否有违反规则
  - 还可以：将这段时间内的各项统计数据获取

| 函数：开始检查                           | 作用                    |
|-----------------------------------|-----------------------|
| ChkStart_MsgAbsCycleTimeViolation | 开始对报文发送周期是否在指定时间范围的检查 |
| ChkStart_SignalValueChange        | 开始对信号是否稳定（不发生变化）的检查   |

| 函数：查询探测结果                     | 作用                     |
|-------------------------------|------------------------|
| ChkQuery_NumEvents            | 查询发生了违反检查规则的事件次数       |
| ChkQuery_StatNumProbes        | 查询探测到的报文的发送次数，或信号变化的次数 |
| ChkQuery_StatProbeIntervalAvg | 查询探测到的报文的发送间隔的平均时长     |
| ChkQuery_StatProbeIntervalMax | 查询探测到的报文的发送间隔的最大时长     |
| ChkQuery_StatProbeIntervalMin | 查询探测到的报文的发送间隔的最小时长     |
| ChkQuery_EventSignalValueMax  | 查询检查期间的信号最大值           |
| ChkQuery_EventSignalValueMin  | 查询检查期间的信号最小值           |

## 六、案例

### 1. 测试ECU（车身控制模块BCM）的功能

#### (1) 需求：测试BCM控制前雨刮的功能是否正确

- 前雨刮（Front Wiper）可能处于四种状态：关闭中、高速刮中、低速刮中、故障中
- BCM模块会处理总线上收到的控制前雨刮工作的请求报文，一旦收到报文，就进行控制处理，并实时反馈
- BCM模块会实时将前雨刮的状态反馈到汽车总线上，持续发送ID为0x387的报文
  - 其中有一个信号 `BCM_FrontWiperStatus`，它的取值如下：
    - 0x0=OFF； 0x1=Low； 0x2=High； 0x3=Error
- 用户可以通过车机/中控来控制前雨刮的工作，本质上就是发送控制前雨刮的请求报文，这个报文如下
  - ID为 `0x55D` 的报文，其中一个信号 `HU_FronWiperRq` 负责控制前雨刮，它的取值如下：



- 0x0 =关闭； 0x1 =低速刮； 0x2 =高速刮； 0x3=aotu档

**(2) 分析：**按照请求信号的规格描述，分别发送不同的控制前雨刮工作的信号值的报文，等待BCM进行处理，并验证前雨刮工作状态的信号（BCM持续在总线上发的）是否符合预期。

- 发送请求控制雨刮的报文（0x55D），查看验证雨刮状态的报文（0x387）

### **(3) 实现：使用CANoe中的Test Module来实现前雨刮控制器（BCM）的功能**

- 创建一个前雨刮功能验证的测试模块 —— 前雨刮功能测试模块
- 在测试模块下创建4个用例函数，分别代表请求4种不同档位的操作

```

/*@!Encoding:936*/
variables
{
 // 进行前雨刮控制的请求报文
 message HU_0x55d msgReq;
 // 检查结果的标识
 int checkResult;
}

MainTest()
{
 testModuleTitle("前雨刮功能测试");
 testModuleDescription("详细测试前雨刮各个档位的工作状态，包括：关闭、高速刮、低速刮、自动挡");

 testReportAddEngineerInfo("测试员", "贾老师");
 testReportAddEngineerInfo("工号", "666888");

 testReportAddSUTInfo("ECU", "BCM（车身控制模块）");
 testReportAddSUTInfo("软件版本", "V1.3.2");

 FrontWiperLow();
 FrontWiperHigh();
 FrontWiperOff();
 FrontWiperAuto();
}

testcase FrontWiperLow()
{
 testCaseTitle("用例1", "测试前雨刮 —— 低速刮");
 testCaseDescription("发送低速刮的请求，验证前雨刮状态信号是否变为低速刮");

 testStep("用例开始时", "请求发送前，当前的工作状态信号BCM_FrontWiperStatus为：0x%X", (int)$BCM_FrontWiperStatus);

 // 设置请求的信号值为1，代表希望控制前雨刮低速刮，并发送该报文
 msgReq.HU_FronWiperRq = 0x1;
 output(msgReq);

 // 检查BCM操作了前雨刮后，更新前雨刮工作状态的信号值为0x1，代表当前前雨刮正在低速刮
 checkResult = testWaitForSignalMatch(BCM_FrontWiperStatus, 1, 500);
 if(checkResult == 1) // 满足检查的条件（前雨刮工作状态的信号值更新为1了）
 {
 testStepPass("用例执行成功", "请求发送后预期值[0x01]，当前的前雨刮工作状态信号BCM_FrontWiperStatus为：0x%02X",
(int)$BCM_FrontWiperStatus);
 }
 else

```

```

{
 testStepFail("用例执行失败", "请求发送后预期值[0x01], 当前的前雨刮工作状态信号BCM_FrontWiperStatus为: 0x%02X",
(int)$BCM_FrontWiperStatus);
}
}

testcase FrontWiperHigh()
{
 testCaseTitle("用例2", "测试前雨刮 —— 高速刮");
 testCaseDescription("发送高速刮的请求, 验证前雨刮状态信号是否变为低高速刮");

 testStep("用例开始时", "请求发送前, 当前的工作状态信号BCM_FrontWiperStatus为: 0x%X", (int)$BCM_FrontWiperStatus);

 // 设置请求的信号值为2, 代表希望控制前雨刮高速刮, 并发送该报文
 msgReq.HU_FronWiperRq = 0x2;
 output(msgReq);

 // 检查BCM操作了前雨刮后, 更新前雨刮工作状态的信号值为0x2, 代表当前前雨刮正在高速刮
 checkResult = testWaitForSignalMatch(BCM_FrontWiperStatus, 2, 500);
 if(checkResult == 1) // 满足检查的条件 (前雨刮工作状态的信号值更新为2了)
 {
 testStepPass("用例执行成功", "请求发送后预期值[0x02], 当前的前雨刮工作状态信号BCM_FrontWiperStatus为: 0x%02X",
(int)$BCM_FrontWiperStatus);
 }
 else
 {
 testStepFail("用例执行失败", "请求发送后预期值[0x02], 当前的前雨刮工作状态信号BCM_FrontWiperStatus为: 0x%02X",
(int)$BCM_FrontWiperStatus);
 }
}

testcase FrontWiperOff()
{
 testCaseTitle("用例3", "测试前雨刮 —— 关闭");
 testCaseDescription("发送关闭的请求, 验证前雨刮状态信号是否变为低关闭");

 testStep("用例开始时", "请求发送前, 当前的工作状态信号BCM_FrontWiperStatus为: 0x%X", (int)$BCM_FrontWiperStatus);

 // 设置请求的信号值为0, 代表希望控制前雨刮关闭, 并发送该报文
 msgReq.HU_FronWiperRq = 0x0;
 output(msgReq);

 // 检查BCM操作了前雨刮后, 更新前雨刮工作状态的信号值为0x0, 代表当前前雨刮已关闭
 checkResult = testWaitForSignalMatch(BCM_FrontWiperStatus, 0, 500);
 if(checkResult == 1) // 满足检查的条件 (前雨刮工作状态的信号值更新为0了)
 {
 testStepPass("用例执行成功", "请求发送后预期值[0x00], 当前的前雨刮工作状态信号BCM_FrontWiperStatus为: 0x%02X",
(int)$BCM_FrontWiperStatus);
 }
 else
 {
 testStepFail("用例执行失败", "请求发送后预期值[0x00], 当前的前雨刮工作状态信号BCM_FrontWiperStatus为: 0x%02X",
(int)$BCM_FrontWiperStatus);
 }
}

testcase FrontWiperAuto()
{
 testCaseTitle("用例4", "测试前雨刮 —— 自动挡");

```

```

testCaseDescription("发送自动挡的请求，验证前雨刮状态信号是在1和2之间");

testStep("用例开始时”，”请求发送前，当前的工作状态信号BCM_FrontWiperStatus为： 0x%X”，(int)$BCM_FrontWiperStatus);

// 设置请求的信号值为3，代表希望控制前雨刮为自动挡，并发送该报文
msgReq.HU_FronWiperRq = 0x3;
output(msgReq);

// 检查BCM操作了前雨刮后，更新前雨刮工作状态的信号值为0x1或者0x2，代表当前前雨刮处于自动挡
checkResult = testWaitForSignalInRange(BCM_FrontWiperStatus, 1, 2, 500);
if(checkResult == 1) // 满足检查的条件（前雨刮工作状态的信号值更新为1或2了）
{
 testStepPass("用例执行成功”，”请求发送后预期值[0x01~0x02]，当前的前雨刮工作状态信号BCM_FrontWiperStatus为： 0x%02X”，(int)$BCM_FrontWiperStatus);
}
else
{
 testStepFail("用例执行失败”，”请求发送后预期值[0x01~0x02]，当前的前雨刮工作状态信号BCM_FrontWiperStatus为： 0x%02X”，(int)$BCM_FrontWiperStatus);
}
}

```

## 2. 测试ECU（车身控制模块BCM）发送报文周期的性能

### (1) 需求：测试一段时间内，BCM发送0x28B报文的间隔时间是否在37到43ms之间

- 矩阵表中标准是40ms，由于ECU的性能，或多或少都会有偏差

### (2) 分析：使用“检查探测”相关的函数

- 先：调用 `ChkStart_MsgAbsCycleTimeViolation` 开启检查
- 再：让程序等待一段时间（例如6000ms），调用 `TestWaitForTimeout`
- 然后：查询检查的各项结果：违规次数、报文探测到次数、报文的平均间隔时间、.....
- 最后：判定“违规次数”决定用例是否通过（`TestStepPass`、`TestStepFail`），并记录统计数据到测试报告中（`TestStep`）
- 额外：可以使用“为检查添加条件”的方式（`TestAddCondition`），对一段时间内发生的每一次违规，都进行判决并加入测试报告。

```

/*@!Encoding:936*/
includes
{

}

variables
{
 const dword gCHECK_TIMEOUT = 6000;
 const word gMIN_INTERVAL = 37;
 const word gMAX_INTERVAL = 43;
}

void MainTest ()
{
 testModuleTitle("测试BCM_0x28B报文的发送性能");
}

```

```

 TestMsgSendInterval();
}

testcase TestMsgSendInterval()
{
 dword checkId;
 long numEvents;
 long statNumProbes;
 double msgIntervalAvg;
 double msgIntervalMin;
 double msgIntervalMax;

 testCaseTitle("用例1", "测试BCM_0x28B报文的发送时间间隔在规定的范围之内");

 // 创建并开启了一次针对报文的发送时间间隔是否在指定范围之内的检查
 // 返回一个检查ID, 以供后续查询检查结果、获取统计数据使用
 checkId = ChkStart_MsgAbsCycleTimeViolation(BCM_0x28b, gMIN_INTERVAL, gMAX_INTERVAL);

 // 为检查添加条件, 记录检查过程中违规的事件信息, 并加入测试报告中
 TestAddCondition(checkId);

 // 持续检查6秒
 testWaitForTimeout(gCHECK_TIMEOUT);

 // 为该检查移除条件
 testRemoveCondition(checkId);

 /*****
 查询检查的各项结果
 查看违规次数 —— ChkQuery_NumEvents
 查看报文出现次数/信号变化次数 —— ChkQuery_StatNumProbes
 查询报文发送间隔的平均时间 —— ChkQuery_StatProbeIntervalAvg
 查询报文发送间隔的最小时间 —— ChkQuery_StatProbeIntervalMin
 查询报文发送间隔的最大时间 —— ChkQuery_StatProbeIntervalMax
 *****/
 // 查询违反检查规则的事件的发生次数, 本例中就是两次报文发送间隔时间不在MIN~MAX毫秒之间
 numEvents = ChkQuery_NumEvents(checkId);
 // 查询报文的出现次数
 statNumProbes = ChkQuery_StatNumProbes(checkId);
 // 查询报文发送间隔的平均时间
 msgIntervalAvg = ChkQuery_StatProbeIntervalAvg(checkId);
 // 查询报文发送间隔的最小时间
 msgIntervalMin = ChkQuery_StatProbeIntervalMin(checkId);
 // 查询报文发送间隔的最大时间
 msgIntervalMax = ChkQuery_StatProbeIntervalMax(checkId);

 if(numEvents == 0) // 没有出现违反规则的时间 (本例中所有探测的报文发送的间隔时间均规定范围内)
 {
 testStepPass("用例通过", "报文发送间隔时间均在在%d~%d毫秒范围", gMIN_INTERVAL, gMAX_INTERVAL);
 }
 else
 {
 testStepFail("用例失败", "违反规则【报文发送间隔时间不在%d~%d毫秒范围】的事件发生次数: %d", gMIN_INTERVAL,
gMAX_INTERVAL, numEvents);
 }

 testStep("统计信息", "报文出现的次数: %d", statNumProbes);
 testStep("统计信息", "报文发送间隔的平均时间: %.3f", msgIntervalAvg);
 testStep("统计信息", "报文发送间隔的最小时间: %.3f", msgIntervalMin);

```

```
testStep("统计信息", "报文发送间隔的最大时间: %.3f", msgIntervalMax);
}
```

## 七、ECU诊断功能的自动化测试

### 1. 诊断的脚本编写的基本思路

1. 先创建诊断请求变量（使用diagRequest关键字，后跟诊断命令名，最后跟变量名）
  - 诊断命令名可以从CAPL Browser右侧的Symbolics面板中复制
2. 发送诊断请求（ `diagSendRequest` ）
3. 等待请求发送完毕（ `testWaitForDiagRequestSent` ）、等待接收到响应（ `testWaitForDiagResponse` ）
4. 获取诊断响应的响应码（`diagGetLastResponseCode`）

```
// 创建了一个进入扩展会话的“诊断请求变量”
diagRequest DoorFL.ExtendedDiagnosticSession_Start reqExtendedSession;
// 创建一个变量代表诊断响应返回的响应码
long respCode;

// 发送诊断请求
diagSendRequest(reqExtendedSession);

// 等待请求发送完毕
if(testWaitForDiagRequestSent(reqExtendedSession, 1000) == 1)
{
 write("进入扩展会话的请求发送完毕");
}
else
{
 write("进入扩展会话的请求发送失败");
 return;
}

// 等待收到诊断响应
if(testWaitForDiagResponse(reqExtendedSession, 1000) == 1)
{
 write("进入扩展会话的响应已经收到");
}
else
{
 write("进入扩展会话的响应未收到");
 return;
}

// 获取诊断响应的响应码
respCode = diagGetLastResponseCode(reqExtendedSession);
if(respCode == -1) // 肯定响应
{
 write("进入扩展会话的响应为肯定响应");
}
else if(respCode == 0) // 未收到响应
{
 write("进入扩展会话的响应还没收到");
}
else
```

```

{
 write("进入会展会话的响应为否定响应，NRC为：0x%02X", respCode);
}

```

## 2. CAPL针对ECU诊断功能提供的常用函数

- 基本流程的函数

| 函数名                        | 作用                                                                                                 |
|----------------------------|----------------------------------------------------------------------------------------------------|
| DiagSetTarget              | 设置后续诊断请求通信的目标ECU（CDD的ECU限定名）                                                                       |
| DiagSendRequest            | 发送指定的诊断请求                                                                                          |
| TestWaitForDiagRequestSent | 等待直到指定的诊断请求已发送                                                                                     |
| TestWaitForDiagResponse    | 等待直到接收到指定诊断请求的响应                                                                                   |
| DiagGetLastResponseCode    | 返回指定诊断请求的最后收到的响应码<br>返回值 = -1，表示收到的是肯定响应<br>返回值 = 0，表示还没有收到响应<br>返回值 > 0，表示收到否定响应，且返回值就是NRC（否定响应码） |

- 设置请求数据、获取响应数据的函数

| 函数名：设置请求数据           | 作用                 |
|----------------------|--------------------|
| DiagSetParameter     | 设置请求中指定参数的物理值      |
| DiagSetParameterRaw  | 设置请求中指定参数的原始字节数组   |
| DiagSetPrimitiveByte | 设置请求报文中的指定索引的单个字节值 |
| DiagSetPrimitiveData | 设置请求报文的原始字节数组      |

| 函数名：获取响应数据                  | 作用                    |
|-----------------------------|-----------------------|
| DiagGetRespParameter        | 获取响应中指定参数的物理值         |
| DiagGetRespParameterRaw     | 获取响应中指定参数的原始字节数组      |
| DiagGetRespPrimitiveSize    | 获取响应报文的字节数            |
| DiagGetRespPrimitiveByte    | 获取响应报文的原始字节数组         |
| DiagGetResplterationCount   | 获取响应中指定的复合参数的值的个数     |
| DiagGetComplexRespParameter | 获取响应中指定的复合参数中指定索引的参数值 |

- 根据种子计算出Key的函数

| 函数名                     | 作用                     |
|-------------------------|------------------------|
| diagGenerateKeyFromSeed | 根据种子值计算出Key（依赖安全DLL文件） |

- 将诊断响应的信息输出的测试报告中

| 函数名                         | 作用              |
|-----------------------------|-----------------|
| testReportWriteDiagResponse | 向测试报文中写入诊断响应的信息 |

### 3. ECU诊断测试的自动化脚本案例

- 思路：结合 TestModule + 诊断的函数

#### 3.1 完成10服务（切换会话的功能）的测试

- 测试用例1：切换到默认会话
- 测试用例2：从默认会话切换到扩展会话
- 测试用例3：从扩展会话切换到编程会话

#### 3.2 完成写入、读取ECU序列号的诊断功能的测试

- 测试用例：
  - 步骤1：切换到扩展会话、解锁ECU
  - 步骤2：写入ECU序列号
  - 步骤3：读取ECU序列号并验证是否与预期一致